

Answers to the Mid-term exam of Information Retrieval

Christian Rinderknecht

27 October 2005

1. Give a version of the naive search algorithm where the comparisons are made from right to left, i.e., the first attempt at matching is done with the word at the end of the text (then slided to the left in case of failure) and the letters are compared from right to left in the text and the word.

RIGHT-LEFT-NAIVE(x, t)

```
1   $i \leftarrow m; j \leftarrow n$ 
2  while  $1 \leq i$  and  $1 \leq j$ 
3      do if  $t[j] = x[i]$ 
4          then  $j \leftarrow j - 1; i \leftarrow i - 1$ 
5          else  $j \leftarrow j + m - i - 1; i \leftarrow m$ 
6  if  $i = 0$ 
7      then ...  $\triangleright$  Occurrence of  $x$  in  $t$  at position  $j + 1$ .
8      else ...  $\triangleright$  No occurrences.
```

2. Give the Morris-Pratt algorithm, assuming you know the supply function.

MP(x, t)

```
1   $s \leftarrow \text{MP-SUPPLY}(x)$ 
2   $i \leftarrow 1; j \leftarrow 1$ 
3  while  $i \leq m$  and  $j \leq n$ 
4      do if  $i = 0$  or  $t[j] = x[i]$ 
5          then  $i \leftarrow i + 1; j \leftarrow j + 1$ 
6          else  $i \leftarrow s[i]$ 
7  if  $i > m$ 
8      then ...  $\triangleright$  Occurrence of  $x$  in  $t$  at position  $j - m$ .
9      else ...  $\triangleright$  No occurrences.
```

3. Give the formal definition of the maximum border of a word.

$$\begin{aligned} \text{BORDER}(a) &= \epsilon \\ \text{BORDER}(y \cdot a) &= \begin{cases} \text{BORDER}(y) \cdot a & \text{if } \text{BORDER}(y) \cdot a \preceq y \\ \text{BORDER}(\text{BORDER}(y) \cdot a) & \text{otherwise} \end{cases} \end{aligned}$$

where $x \preceq y$ means “ x is a prefix of y ”, a is a letter and y is a non-empty word.

4. Use this definition to compute the maximum borders of all the prefixes of the word **acabbacabb**.

$$\begin{aligned} \text{BORDER}(a) &= \epsilon \\ \text{BORDER}(a \cdot c) &= \text{BORDER}(\text{BORDER}(a) \cdot c) \end{aligned}$$

because $\text{BORDER}(a) \cdot c = \epsilon \cdot c = c \not\preceq a$. Thus

$$\text{BORDER}(ac) = \text{BORDER}(c) = \epsilon$$

Now:

$$\text{BORDER}(ac \cdot a) = \text{BORDER}(ac) \cdot a$$

since $\text{BORDER}(ac) \cdot a = \epsilon \cdot a = a \preceq ac$. Hence

$$\text{BORDER}(aca) = a$$

Now:

$$\text{BORDER}(aca \cdot b) = \text{BORDER}(\text{BORDER}(aca) \cdot b)$$

because $\text{BORDER}(aca) \cdot b = ab \not\preceq aca$. Therefore

$$\text{BORDER}(acab) = \text{BORDER}(ab)$$

We have to compute $\text{BORDER}(ab)$:

$$\text{BORDER}(a \cdot b) = \text{BORDER}(\text{BORDER}(a) \cdot b)$$

since $\text{BORDER}(a) \cdot b = \epsilon \cdot b = b \not\preceq a$. Thus

$$\begin{aligned} \text{BORDER}(acab) &= \text{BORDER}(ab) \\ &= \text{BORDER}(b) \\ &= \epsilon \end{aligned}$$

Now

$$\text{BORDER}(acab \cdot b) = \text{BORDER}(\text{BORDER}(acab) \cdot b)$$

because $\text{BORDER}(acab) \cdot b = \epsilon \cdot b = b \not\preceq acab$. Then

$$\text{BORDER}(acabb) = \text{BORDER}(b) = \epsilon$$

Now

$$\text{BORDER}(acabb \cdot a) = \text{BORDER}(acabb) \cdot a$$

since $\text{BORDER}(acabb) \cdot a = \epsilon \cdot a = a \preceq acabb$. Therefore

$$\text{BORDER}(acabba) = a$$

Now

$$\text{BORDER}(acabba \cdot c) = \text{BORDER}(acabba) \cdot c$$

because $\text{BORDER}(acabba) \cdot c = a \cdot c = ac \preceq acabba$. Thus

$$\text{BORDER}(acabbac) = ac$$

Now

$$\text{BORDER}(acabbac \cdot a) = \text{BORDER}(acabbac) \cdot a$$

since $\text{BORDER}(acabbac) \cdot a = ac \cdot a = aca \preceq acabbac$. Hence

$$\text{BORDER}(acabbaca) = aca$$

Now

$$\text{BORDER}(acabbaca \cdot b) = \text{BORDER}(acabbaca) \cdot b$$

because $\text{BORDER}(acabbaca) \cdot b = aca \cdot b = acab \preceq acabbaca$. Then

$$\text{BORDER}(acabbacab) = acab$$

Finally

$$\text{BORDER}(acabbacab \cdot b) =$$

since $\text{BORDER}(acabbacab) \cdot b = acab \cdot b = acabb \preceq acabbacab$. Thus

$$\text{BORDER}(acabbacabb) = acabb$$

5. Let the word **abacabac** and the text **babacacabacaab**. Show the different positions of the word with respect to the text during a run of the Morris-Pratt algorithm (show the failure comparisons by underlining the pairs of different letters).

	1	2	3	4	5	6	7	8	9	10	11	12	13	14
$t :$	b	a	b	a	c	a	c	a	b	a	c	a	a	b

$x :$	a	b	a	c	a	b	a	c						
		a	b	a	c	a	b	a	c					
					a	b	a	c	a	b	a	c		
						a	b	a	c	a	b	a	c	
							a	b	a	c	a	b	a	c

Note: the failures are coloured in grey.